Design Project - MirrorWalk

MirrorWalk

A Native Android Application for Spatially Anchored Ghost-Race Running and Route Tracking

Submitted by:

1024160103 Ana Hasan
1024160117 Agrim Gupta
1024160108 Janvi Verma
1024160125 Yashkaran Singh

Group: 9 BE Third Year, CSE

Submitted to:

Anushka Ma'am

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **5**
- 1.1 Background and Context . . . 5
  - 1.1.1 Problem Statement . . . 5
- 1.2 Project Objectives . . . 5

2 Methodology and Design **7**
- 2.1 System Architecture . . . 7
  - 2.1.1 High-Level Design . . . 7
  - 2.1.2 Use Case Diagram . . . 8
  - 2.1.3 Activity Diagram . . . 9
  - 2.1.4 Technical Stack . . . 10
- 2.2 Implementation Details . . . 10
  - 2.2.1 Class Diagram . . . 10
  - 2.2.2 Sequence Diagrams . . . 11
  - 2.2.3 Module 1: Run and Walk Tracking . . . 12
  - 2.2.4 Module 2: Run History and Local Storage . . . 13
  - 2.2.5 Module 3: AR Ghost-Runner (Prototype Stage) . . . 13
  - 2.2.6 Visual Design . . . 14

3 Results and Evaluation **15**
- 3.1 Testing Methodology . . . 15
- 3.2 Performance Metrics . . . 15
- 3.3 Analysis . . . 16

4 Conclusion **17**
- 4.1 Summary of Work . . . 17
- 4.2 Key Findings . . . 17
- 4.3 Future Work and Recommendations . . . 17
- 4.4 Final Remarks . . . 18

Bibliography **19**

Chapter 1

Introduction

1.1 Background and Context

Running and walking are two of the most popular forms of exercise because they need almost no equipment and can be done almost anywhere. Over the last few years, fitness tracking apps have become very common, but most of them focus only on numbers: distance, pace, and calories. They rarely make running feel social or fun the way apps like Instagram make photo-sharing fun, and they rarely give the user something exciting to race against.

At the same time, Augmented Reality (AR) has become much more accessible on regular smartphones through frameworks like Google ARCore, which means it is now possible to place virtual objects into the real world using just a phone camera, without any special hardware. This opens the door to combining fitness tracking with AR in a way that was not practical before.

MirrorWalk is our attempt to bring these two ideas together: a social network for runners, similar in spirit to Instagram, combined with an AR-based "ghost runner" that lets a user race against a recording of a past run (their own or someone else's) overlaid onto the real world through their phone camera.

1.1.1 Problem Statement

Existing running and fitness apps generally fall into two categories. The first category is purely data-driven: they track distance, pace, and route, but offer little social interaction or motivation beyond a leaderboard or a badge. The second category is purely social: they let users share photos and updates but do not track detailed run data or offer any interactive way to compete against past performances.

There is no widely available app that combines all three of the following in one place: (1) accurate live GPS-based run tracking, (2) an Instagram-style social feed built specifically around running, and (3) an augmented-reality feature that lets a runner visually race against a "ghost" of a previous run in real time. MirrorWalk is designed to fill this gap. The problem we are solving is the lack of an engaging, social, and visually motivating way to track and improve running performance.

1.2 Project Objectives

1. **Objective 1:** Build a reliable run/walk tracking system that records GPS location, distance, pace, and time accurately, while filtering out noisy or unrealistic

GPS readings, so that users get a trustworthy record of their activity.

2. **Objective 2:** Build a social layer around running, similar to Instagram, where users have profiles, follower and following counts, and can post their runs along with optional photos, so that running becomes a shared and motivating experience rather than a solitary activity.

3. **Objective 3:** Build an AR "ghost runner" feature that uses ARCore to place a virtual runner into the real world through the phone camera, moving according to a previously recorded run, so that a user can visually race against a past performance and compare their live pace and distance against it in real time.

These objectives are Specific (each targets one clear feature area), Measurable (tracking accuracy, working AR anchoring, and a working social feed can all be checked), Achievable (all required technologies such as ARCore, Fused Location Provider, and Jetpack Compose are freely available and well documented), Relevant (they directly support the app's core idea of social and AR-enhanced running), and Time-bound (they are being delivered in stages, starting with the prototype already built and continuing through future milestones).

Chapter 2

Methodology and Design

2.1 System Architecture

MirrorWalk is being built as a native Android application. At a high level, the app is organised into three main functional areas that work together:

1. **Tracking Engine** – responsible for requesting location permissions, listening to GPS updates, filtering bad readings, and calculating live distance, pace, and elapsed time during a run.

2. **Social Layer** – responsible for user profiles, follower/following relationships, and posts that show a completed run along with optional photos and fitness statistics.

3. **AR Ghost-Runner Module** – responsible for taking a recorded run (the "ghost"), placing a translucent 3D runner model into the real-world camera view using ARCore, and moving that model over time to match the recorded run, while comparing it live against the current user's run.

2.1.1 High-Level Design

The app follows a layered architecture that is standard for modern Android apps built with Jetpack Compose:

- **UI Layer (Jetpack Compose):** Screens such as the run screen, the map screen, the run history screen, and the AR ghost-run screen are all built as composable functions.

- **ViewModel Layer:** Each screen is backed by an Android ViewModel that holds UI state using Kotlin `StateFlow`. The ViewModel exposes the current distance, pace, elapsed time, and route points to the UI, and reacts to user actions like Start, Pause, Resume, and Stop.

- **Data Layer:** Location updates come from the Fused Location Provider. Completed runs are saved locally using Room, so that a user's run history is available even without an internet connection.

- **Rendering Layer:** The live route is drawn on a MapLibre map using OpenFreeMap tiles, and the ghost runner is rendered in AR using SceneView with Filament, on top of ARCore's plane detection.

This separation keeps the tracking logic independent of how it is displayed, which makes it easier to test the GPS filtering and distance/pace calculations on their own, without needing the full UI running.

2.1.2 Use Case Diagram

Figure 2.1 shows the main actions a user can perform in MirrorWalk, grouped by feature area (run tracking, AR ghost runner, and the planned social features), along with the external systems the app depends on, such as the Fused Location Provider, Room database, MapLibre/OpenFreeMap, and ARCore/SceneView.

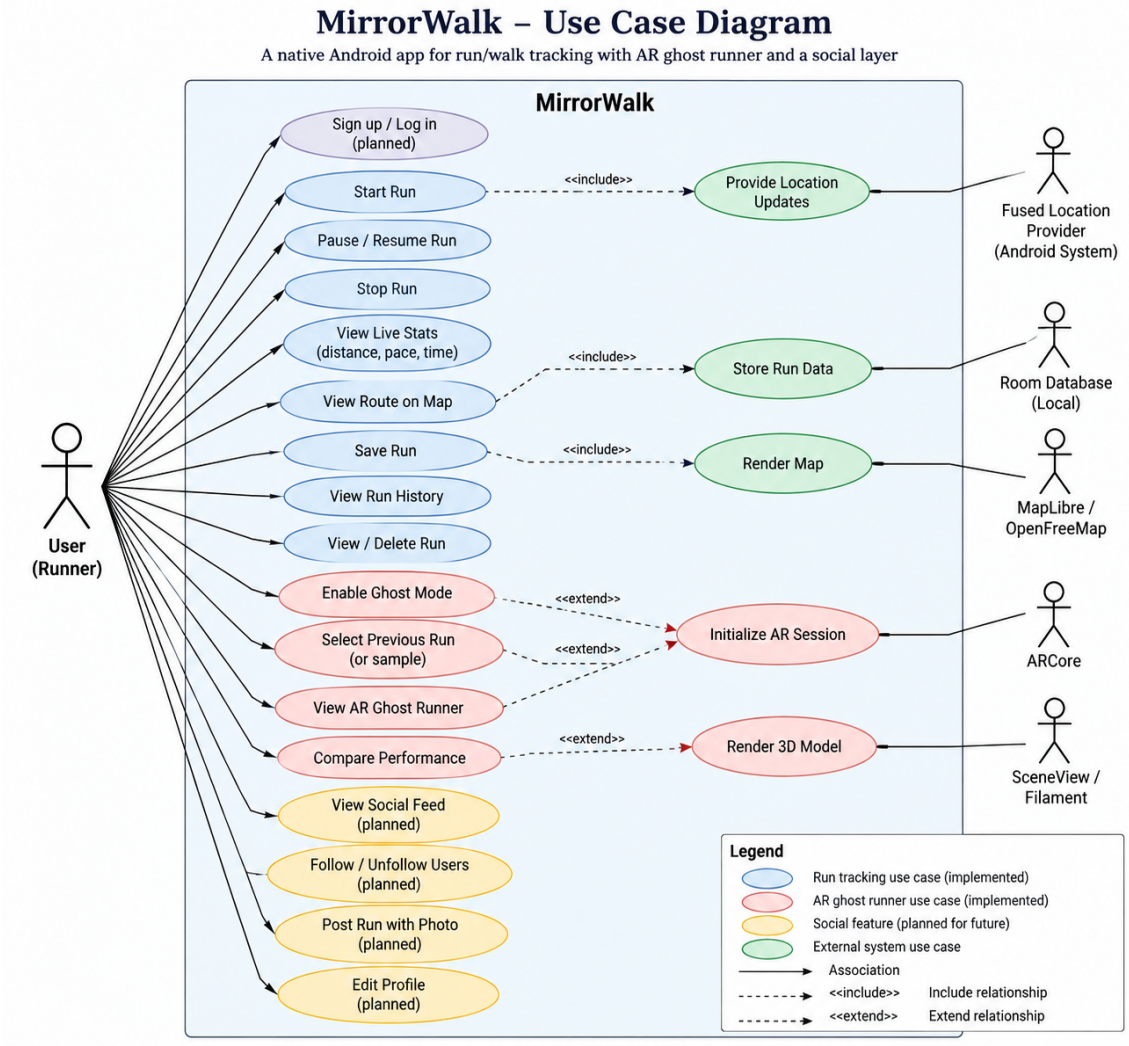


Figure 2.1: Use Case Diagram of MirrorWalk

2.1.3 Activity Diagram

Figure 2.2 shows the complete workflow of the app, from launching the app and requesting location permission, through live run tracking, to the optional AR ghost-runner flow and finally saving the run and viewing run history.

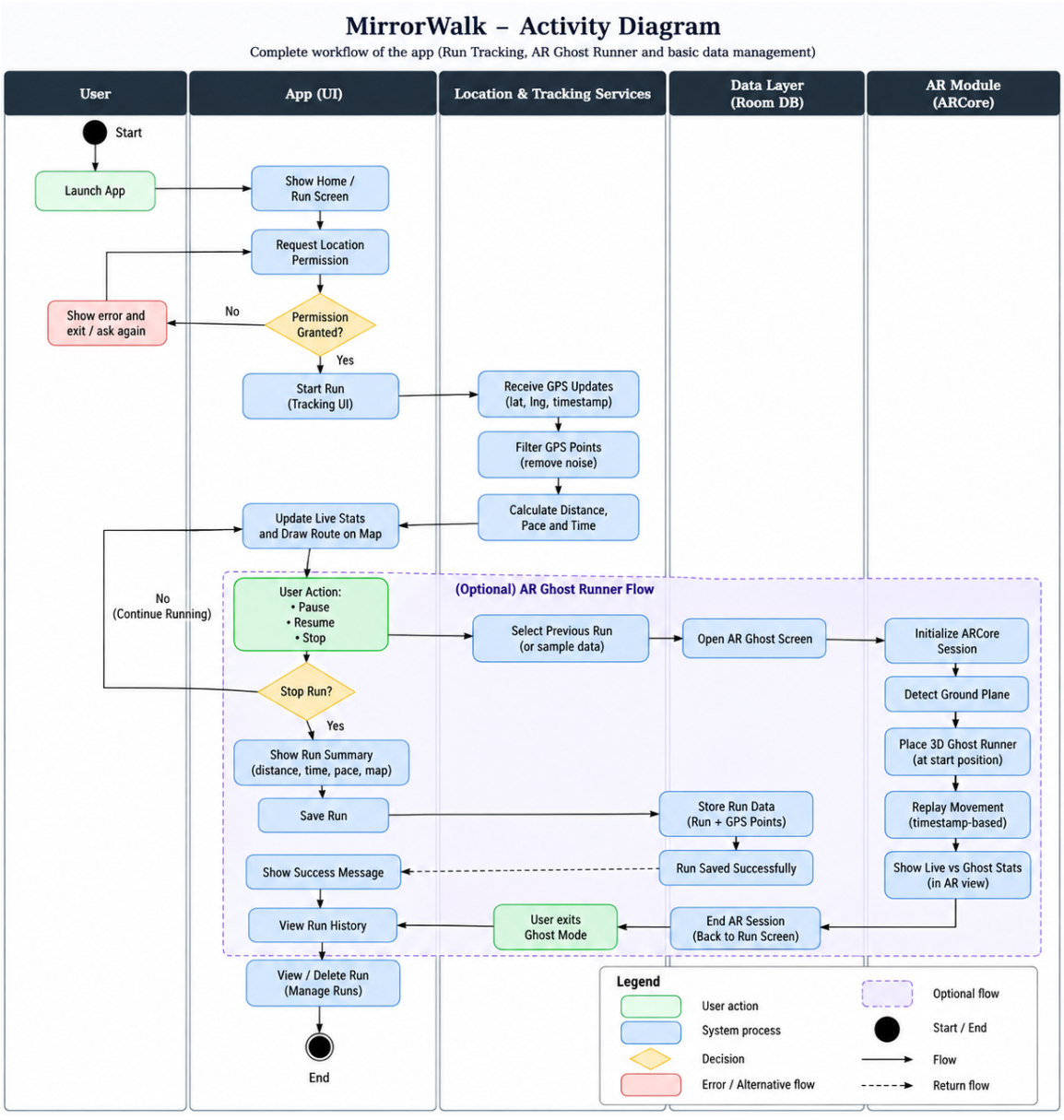


Figure 2.2: Activity Diagram of MirrorWalk

2.1.4 Technical Stack

- **Language:** Kotlin 2.4.x [Kot26]
- **IDE:** Android Studio Quail 4 / 2026.1.4
- **Build system:** Gradle 9.6
- **Android build plugin (AGP):** 9.4.0
- **Java runtime:** JDK 17
- **UI framework:** Jetpack Compose with Material 3 [And26a]
- **Navigation:** Navigation Compose [And26c]
- **State management:** Kotlin Coroutines [And26b], StateFlow, and Android ViewModel [And26e]
- **Location tracking:** Google Play Services Fused Location Provider [Goo26c]
- **Maps:** MapLibre Native Android SDK [Map26] with OpenFreeMap tiles [Ope26] (no Google Maps API key or billing account required)
- **Local database:** Room 2.8.5 [And26d]
- **Augmented Reality:** Google ARCore 1.56.0 [Goo26b]
- **AR/3D rendering:** SceneView AR 4.34.0 [Sce26] with Filament [Goo26a]
- **Permissions used:** Fine location, coarse location, camera, and internet access

We chose MapLibre with OpenFreeMap instead of Google Maps specifically because it removes the need for an API key or a billing account, which keeps the project free to run and easy for any team member to build and test. Similarly, ARCore and SceneView were chosen because they are the most well-supported, actively maintained AR tools for native Android development, and they integrate cleanly with Jetpack Compose.

2.2 Implementation Details

2.2.1 Class Diagram

Figure 2.3 shows how the app's code is organised into layers: the UI layer built with Jetpack Compose, the ViewModel layer that holds state for each screen, the Domain/Model layer (such as `User`, `Run`, `GhostRun`, and `GPSPoint`), the Service/Manager

layer that does the actual work (such as `LocationTracker`, `GPSFilter`, `RunCalculator`, and `ARManager`), and the Data layer backed by Room. Classes marked "(Planned)", such as `Post` and `Comment`, belong to the social feature that has not been implemented yet.

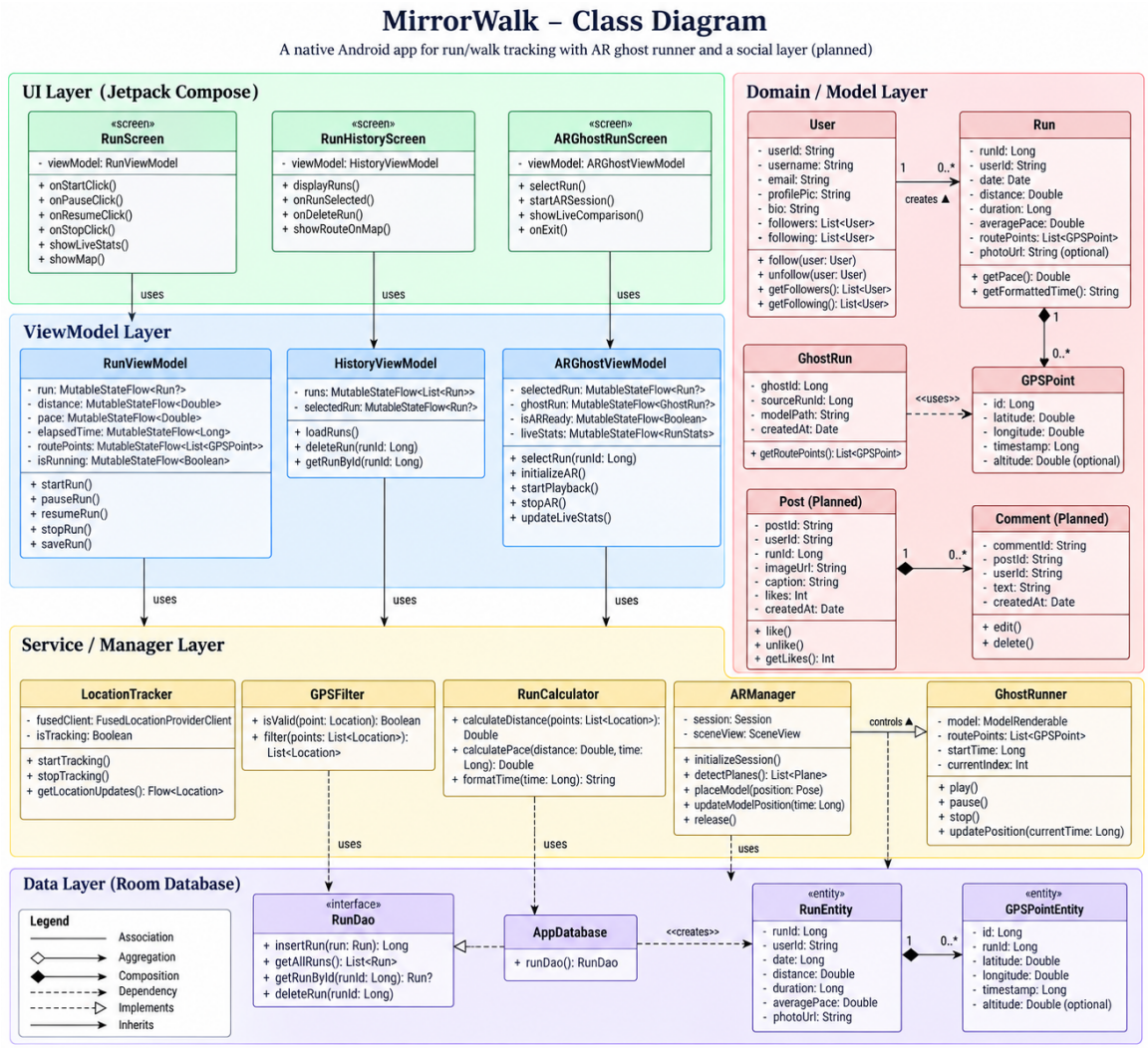


Figure 2.3: Class Diagram of MirrorWalk

2.2.2 Sequence Diagrams

Figure 2.4 shows the exact order of method calls between components for the two core workflows of the app. The first diagram traces the Run Tracking flow: starting a run, requesting location updates, filtering and processing each GPS point in a loop while the run is active, handling pause/resume, and finally saving the completed run to the

Room database. The second diagram traces the AR Ghost Runner flow: selecting a previous run, initialising the ARCore session, detecting ground planes, placing and animating the ghost model based on its recorded timestamps, and showing the live comparison between the user and the ghost until the AR session ends.

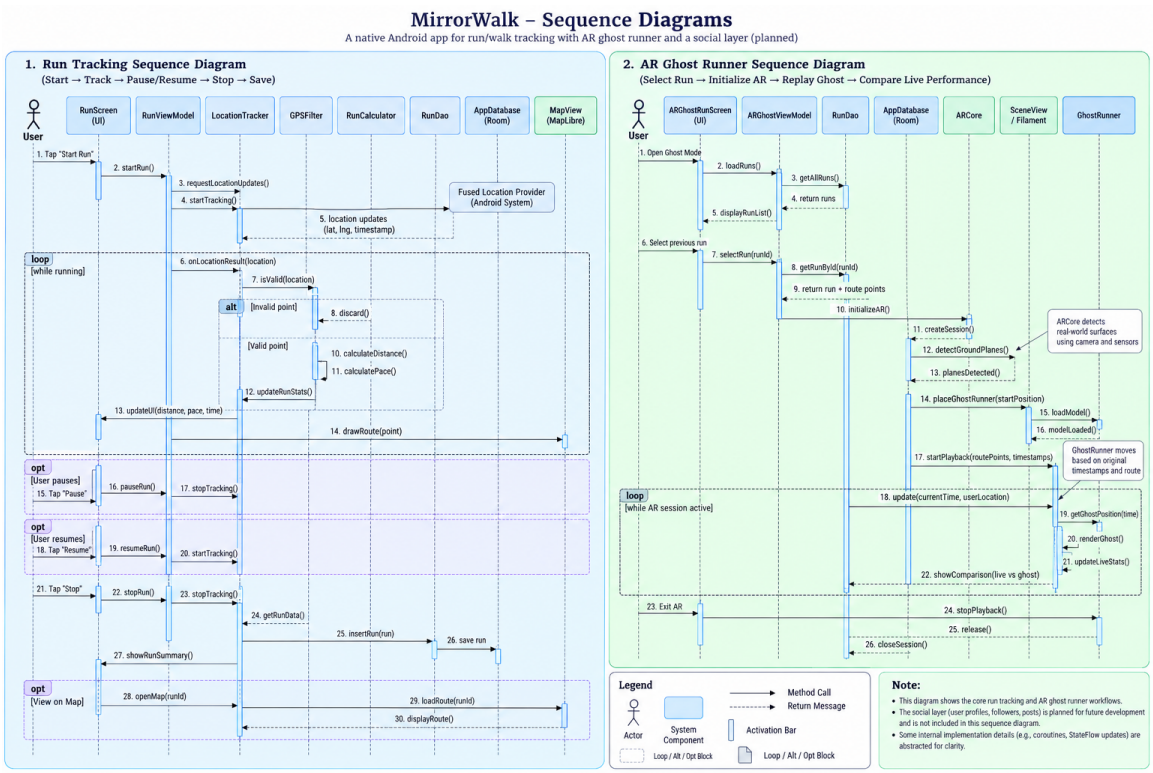


Figure 2.4: Sequence Diagrams of MirrorWalk: Run Tracking (left) and AR Ghost Runner (right)

2.2.3 Module 1: Run and Walk Tracking

This is the core module of the app and is the most complete part of the current prototype. It is responsible for turning raw GPS data into a clean, trustworthy run record.

Session Controls. The user can Start, Pause, Resume, and Stop a run or walk session. These controls change the internal state of the ViewModel, which decides whether new GPS points should be recorded, ignored (while paused), or used to finalise the run (on stop).

Location Permission Handling. When the user presses Start for the first time, the app requests fine and coarse location permission at runtime, following Android's standard permission model. Tracking only begins once permission is granted.

Continuous GPS Tracking. While a session is active, the app subscribes to continuous location updates from the Fused Location Provider. Each update includes latitude, longitude, and a timestamp, which are stored as a single GPS point.

GPS Filtering. Raw GPS data is often noisy, especially in areas with tall buildings or poor signal. Points are filtered out if they show an unrealistic speed (for example, a jump that would mean the user was moving faster than is physically reasonable for a runner), poor location accuracy reported by the device, or jitter where the phone reports small back-and-forth movement while the user is actually stationary. This filtering step is important because it directly affects how correct the final distance and pace numbers are.

Live Metrics. Using the filtered GPS points, the app continuously calculates:

- **Distance**, in kilometres, calculated by adding up the distance between each pair of consecutive valid points.
- **Pace**, in minutes per kilometre, calculated from the distance covered and the time elapsed.
- **Elapsed time**, displayed live and updated every second.

Live Route Drawing. As new valid points come in, they are drawn as a growing line on the map in real time, so the user can see the exact path they have taken so far. Once the session is stopped, the final complete route is displayed on the map for review.

2.2.4 Module 2: Run History and Local Storage

Once a run is stopped, it is saved locally on the device using Room. Each saved run includes the full list of route points, the total distance, and the average pace. A dedicated **Saved Runs History** screen lists all previous runs, and tapping on any entry opens that run's exact route again on the map. Users can also delete or discard a saved route they no longer want to keep. Because this data is stored locally, run history remains available even without an internet connection.

2.2.5 Module 3: AR Ghost-Runner (Prototype Stage)

This is the most experimental and distinctive module of MirrorWalk. The idea is to let a user race against a "ghost" – a visual replay of a previously recorded run – directly overlaid onto the real world using the phone's camera.

Ghost Mode Toggle. Before starting a session, the user can switch on Ghost Mode. In the current prototype, the ghost run is based on hardcoded sample route data, but this data is structured in the same shape as a normal saved run, so it can

later be replaced with any real, previously saved run without changing the rest of the system.

AR Setup. When Ghost Mode is active, the app opens an AR screen built with ARCore and SceneView. ARCore performs plane detection to identify a flat ground surface in the camera view, and the ghost runner (currently a translucent placeholder 3D model) is anchored to that detected ground plane so that it appears to stand on the real ground.

Time-Based Ghost Movement. The ghost runner does not move at a constant speed. Instead, its position at any moment is calculated from the original recorded run's timestamps, so the ghost moves exactly the way the original run happened – speeding up and slowing down at the same points the original runner did.

Live Comparison Overlay. While the AR session is running, an overlay on screen shows the current user's live distance and pace next to the ghost's distance and pace at the same point in time. This gives the user immediate, visual feedback on whether they are ahead of or behind their target run.

2.2.6 Visual Design

MirrorWalk uses a dark navy and charcoal colour scheme with mint and lime accent colours, giving the app an energetic but easy-on-the-eyes look that suits both daytime and low-light use (for example, early morning or night runs). The interface uses rounded cards for content blocks, a bottom navigation bar for moving between the main sections of the app, and a central action button for starting a run, similar in style to popular social and fitness apps. The primary UI font is Manrope, chosen for its clean and modern look at small sizes, while Space Grotesk Bold is used for subtitles and section headers to give extra visual emphasis where needed.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Since MirrorWalk is currently at the prototype stage, testing has focused mainly on the tracking engine and the AR module, as these carry the highest technical risk.

- **Unit Testing:** The distance calculation, pace calculation, and GPS filtering logic (speed checks, accuracy checks, and jitter checks) were tested individually with sample sets of GPS points, including deliberately noisy data, to confirm that bad points are correctly rejected.
- **Integration Testing:** The full flow of Start, Pause, Resume, and Stop was tested together with live location updates and map drawing, to confirm that the UI, ViewModel, and location service stay in sync.
- **Performance Testing:** Live tracking was tested during real outdoor walks and runs to check that the distance and pace values update smoothly without noticeable lag or battery drain issues.
- **AR Testing:** The AR ghost-runner screen was tested on flat outdoor and indoor surfaces to check that ARCore's plane detection reliably anchors the ghost model, and that the ghost's movement matches the timestamps of the recorded route.

3.2 Performance Metrics

The table below summarises the current status of the prototype's core features. Since this is a mid-semester evaluation, the numbers reflect the state of the prototype so far rather than a finished production app.

Feature Area	**Target**	**Current Status**
GPS point filtering	Reduce noisy points	Working
Live distance/pace calc.	Real-time updates	Working
Route saving (Room)	Persist after app close	Working
AR plane detection	Reliable ground anchor	Working (prototype)
Ghost movement accuracy	Match recorded timestamps	Working (hardcoded data)

Table 3.1: Prototype Feature Status Summary

3.3 Analysis

The core run-tracking objective has largely been met: sessions can be started, paused, resumed, and stopped reliably, GPS filtering successfully removes clearly incorrect points, and live distance, pace, and route drawing all work as intended. Local storage using Room means users do not lose their run history between app launches, which was an important requirement.

The AR ghost-runner feature, while still early, has exceeded our initial expectations for this stage: we already have working plane detection, ground anchoring, and time-based movement, which are usually the hardest parts of an AR feature to get right. The main current limitation is that the ghost run is based on hardcoded sample data rather than a real, previously saved run. This was a deliberate choice to isolate and test the AR rendering and movement logic separately from the run-saving logic, before connecting the two together.

The main challenge encountered so far has been GPS noise, especially brief inaccurate readings that could otherwise cause sudden spikes in the calculated distance or pace. This was resolved by adding filtering rules based on maximum realistic speed, GPS accuracy values reported by the device, and jitter detection when the user is not actually moving. Another challenge was reliably anchoring the AR model to a stable position; this was addressed by relying on ARCore's built-in plane detection rather than attempting manual surface estimation.

The social layer (profiles, followers/following, and the post feed) and the fitness analysis feature are planned next steps and are not yet part of the working prototype, so they are not included in the testing results above.

Chapter 4

Conclusion

4.1 Summary of Work

In this stage of the project, we set out to build the foundation of MirrorWalk, an app that combines social running with an AR ghost-runner racing feature. We have successfully built and tested a working run/walk tracking engine, including session controls, GPS filtering, live distance and pace calculation, live route drawing, and local storage of completed runs with a history screen. We have also built an early but functional AR ghost-runner prototype, including plane detection, ground anchoring, time-based ghost movement, and a live comparison overlay. The social feed, follower system, and fitness analysis features are planned for the next stage of development.

4.2 Key Findings

- **Finding 1:** Reliable run tracking depends heavily on good GPS filtering; without filtering rules for speed, accuracy, and jitter, raw GPS data alone produces noticeably incorrect distance and pace values.
- **Finding 2:** ARCore's plane detection is accurate and stable enough on modern Android devices to anchor a virtual runner convincingly on real ground, even at this early prototype stage, which was not something we were fully certain of before testing.
- **Finding 3:** Structuring the hardcoded ghost-run data in the same format as a normal saved run made it straightforward to plan how the AR module will later connect to real saved runs, without needing to redesign the AR movement logic.

4.3 Future Work and Recommendations

1. Replace the hardcoded ghost-run data with real, user-selected saved runs, so any past run (the user's own or a friend's) can be used as a ghost.
2. Build the social layer: user profiles, follower and following counts, and a post feed where users can share their runs along with optional photos.
3. Add the fitness analysis feature, showing trends over time such as average pace, weekly distance, and improvement compared to past runs.

4. Improve GPS filtering further, for example by smoothing the route using techniques that account for the user's likely path rather than just raw point-to-point distance.

5. Consider real-world deployment steps, such as battery optimisation for long AR sessions and testing on a wider range of Android devices with different camera and sensor quality.

4.4 Final Remarks

Building MirrorWalk so far has shown us that combining accurate sensor-based tracking with augmented reality is achievable on a regular smartphone, without needing any specialised hardware. The biggest lesson from this stage has been the importance of getting the "boring but essential" parts right first, such as GPS filtering and reliable local storage, before layering more exciting features like AR on top. With the tracking foundation now working, we are confident about extending MirrorWalk into a fully social, AR-powered running app in the next stage of the project.

Bibliography

[And26a] Android Developers. *Jetpack Compose*. Android Developers, Google. 2026. URL: https://developer.android.com/develop/ui/compose (cit. on p. 10).

[And26b] Android Developers. *Kotlin Coroutines on Android*. Android Developers, Google. 2026. URL: https://developer.android.com/kotlin/coroutines (cit. on p. 10).

[And26c] Android Developers. *Navigation with Compose*. Android Developers, Google. 2026. URL: https://developer.android.com/develop/ui/compose/navigation (cit. on p. 10).

[And26d] Android Developers. *Save Data in a Local Database Using Room*. Android Developers, Google. 2026. URL: https://developer.android.com/training/data-storage/room (cit. on p. 10).

[And26e] Android Developers. *ViewModel*. Android Developers, Google. 2026. URL: https://developer.android.com/topic/libraries/architecture/viewmodel (cit. on p. 10).

[Goo26a] Google. *Filament: Real-Time Physically Based Rendering Engine*. GitHub. 2026. URL: https://github.com/google/filament (cit. on p. 10).

[Goo26b] Google Developers. *Geospatial Developer Guide*. ARCore Documentation, Google. 2026. URL: https://developers.google.com/ar/develop/geospatial (cit. on p. 10).

[Goo26c] Google Developers. *LocationServices*. Google Play Services Documentation, Google. 2026. URL: https://developers.google.com/android/reference/com/google/android/gms/location/LocationServices (cit. on p. 10).

[Kot26] Kotlin. *Kotlin Documentation*. JetBrains. 2026. URL: https://kotlinlang.org/docs/home.html (cit. on p. 10).

[Map26] MapLibre. *MapLibre Native Android Developer Documentation*. MapLibre. 2026. URL: https://maplibre.org/maplibre-native/docs/book/platforms/android/ (cit. on p. 10).

[Ope26] OpenFreeMap. *OpenFreeMap Quick Start Guide*. 2026. URL: https://openfreemap.org/quick_start/ (cit. on p. 10).

[Sce26] SceneView. *SceneView: 3D & AR SDK for Android*. GitHub. 2026. URL: https://github.com/SceneView/sceneview (cit. on p. 10).